# Learn Home Automation with Arduino

Lesson 3: Working with Sensors

---

## Working with Sensors – Study Notes

---

### Key Concepts

1. **DHT11 sensor basics** – how the temperature/humidity sensor works and its specifications.
2. **Wiring the DHT11 to an Arduino** – correct pin connections and power considerations.
3. **Using the DHT library** – installing, including, and calling library functions to read data.
4. **Serial monitoring** – printing sensor values to the Serial Monitor for debugging and logging.
5. **Common pitfalls** – timing, pull  up resistors, and handling read errors.

---

### Summary

The DHT11 is an inexpensive, digital temperature and humidity sensor that communicates with an Arduino via a single  wire serial protocol. Because it outputs data as digital pulses rather than an analog voltage, the Arduino does not need an ADC channel; instead, a small library handles the timing  critical bit  banging required to decode the sensor's signal.

To use the DHT11, you connect its VCC (3.3 /V–5 /V), GND, and data pin to the Arduino, optionally adding a 4.7 /k:™c  ö³© pull  up resistor on the data line. After installing the **DHT sensor library** (e.g., "DHT sensor library" by Adafruit), you create a `DHT` object, specify the sensor type (`DHT11`) and the data pin, and call `readTemperature()` and `readHumidity()` inside `loop()`. The values are then printed to the Serial Monitor, where you can verify that the sensor is functioning correctly.

Understanding the timing constraints (the sensor can be read only once every 1–2 /seconds) and handling error returns (NaN) are essential for reliable operation. Once you're comfortable with the DHT11, the same wiring and code structure can be adapted for other digital sensors such as the DHT22, BMP280, or DS18B20.

---

### Important Points

- **Pinout** (view the sensor's front):
  - VCC !' 5 /V (or 3.3 /V)
  - GND !' GND
  - DATA !' any digital pin (commonly D2)
  - **Optional**: 4.7 /k:™c  ö³© pull  up resistor between DATA and VCC.
- **Library installation**: Sketch !' Include Library !' Manage Libraries !' search "DHT sensor library" (by Adafruit) and install.
- **Creating the object**:

```cpp
#include "DHT.h"
```

```
#define DHTPIN 2          // data pin
#define DHTTYPE DHT11     // sensor model
DHT dht(DHTPIN, DHTTYPE);
```

- **Initialization**: Call `dht.begin();` in `setup()`.
- **Reading values**:

```cpp
float h = dht.readHumidity();          // %RH
float t = dht.readTemperature();       // °C
```

- Returns `NaN` on failure; always check with `isnan()`.
- **Timing**: Minimum 1 /second between reads; use `delay(2000);` or a non  blocking timer.
- **Serial output**: `Serial.begin(9600);` in `setup()`, then `Serial.print()`/`println()` in `loop()`.
- **Common errors**:

- Wrong pin number or missing pull  up !' always reads `NaN`.
- Reading too fast !' sensor returns previous value or error.
- Powering from 5 /V is fine, but keep wiring short to reduce noise.

---

## Examples

### Example 1 – Basic DHT11 reading

```cpp
#include "DHT.h"
#define DHTPIN 2          // Connect DATA pin to digital pin 2
#define DHTTYPE DHT11     // DHT 11 sensor
DHT dht(DHTPIN, DHTTYPE);
void setup() {
  Serial.begin(9600);
  dht.begin();
  Serial.println("DHT11 test!");
}
void loop() {
  // Wait a few seconds between measurements
  delay(2000);
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();
  // Check if any reads failed and exit early (to try again later)
  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }
  Serial.print("Humidity: ");
```

```
  Serial.print(humidity);
  Serial.print(" %\t");
  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" *C");
}
```

*Explanation*: The sketch initializes the sensor, then every two seconds reads humidity and temperature. If a read fails, it prints an error message; otherwise, it displays the values on the Serial Monitor.

### Example 2 – Conditional LED indicator

```cpp
#include "DHT.h"
#define DHTPIN 2
#define DHTTYPE DHT11
#define LEDPIN 13          // Built  in LED
DHT dht(DHTPIN, DHTTYPE);
void setup() {
  Serial.begin(9600);
  pinMode(LEDPIN, OUTPUT);
  dht.begin();
}
void loop() {
  delay(2000);
  float temp = dht.readTemperature();
  if (isnan(temp)) {
    Serial.println("Read error");
    return;
  }
  // Turn LED on if temperature exceeds 30 /°C
  if (temp > 30.0) {
    digitalWrite(LEDPIN, HIGH);
    Serial.println("Hot! LED ON");
  } else {
    digitalWrite(LEDPIN, LOW);
    Serial.println("Temp OK");
  }
}
```

*Explanation*: In addition to printing temperature, this sketch lights the onboard LED when the temperature is above 30 /°C, demonstrating how sensor data can drive actuators.

---

## Quick Review

1. What are the three pins on a DHT11 and how should they be connected to an Arduino?
2. Why is a pull  up resistor recommended on the DATA line, and what value is typical?
3. How often may you safely request a new reading from a DHT11?
4. What function do you use to check whether a sensor read returned a valid number?
5. Write a one  sentence description of what `dht.begin()` does.

---

## Further Reading

- **Other DHT models** – DHT22 (higher accuracy, wider range) and wiring differences.
- **I²C and SPI sensors** – BMP280 (pressure/temperature) and how to use the Wire library.
- **Non  blocking timing** – using `millis()` instead of `delay()` for responsive sketches.
- **Data logging** – writing sensor values to an SD card or sending them over Serial/Bluetooth.
- **Calibration & error handling** – techniques for smoothing noisy sensor data (moving average, median filter).